

Lộ trình Backend để tiến tới tư duy Senior

Học cấu trúc hệ thống, giữ invariant, review theo rủi ro và chịu trách nhiệm cho production.

React · FastAPI · MySQL · InfluxDB · MQTT · WebSocket

9 giai đoạn

Từ baseline đến system design và ownership

M0-M4

Đánh giá mastery bằng bằng chứng

90+ câu hỏi

Gate kiểm tra kiến thức và tình huống

Mục lục

1. Mục tiêu và cách sử dụng
2. Hệ đo mastery M0–M4
3. Bản đồ 9 giai đoạn
4. Giai đoạn 0 — Baseline và rủi ro
5. Giai đoạn 1 — Runtime, network, API
6. Giai đoạn 2 — Database và integrity
7. Giai đoạn 3 — Security và identity
8. Giai đoạn 4 — Testing và review
9. Giai đoạn 5 — Messaging và distributed systems
10. Giai đoạn 6 — Observability, SRE, performance
11. Giai đoạn 7 — Delivery, backup, DR
12. Giai đoạn 8 — Architecture và năng lực senior
13. Dashboard tuần/tháng
14. Evidence pack và checklist review
15. Protocol vibe coding an toàn
16. Capstone và mốc tốt nghiệp
17. Danh mục tài liệu cốt lõi

Cách đọc hiệu quả

Không đọc PDF này từ đầu đến cuối như giáo trình. Chọn đúng giai đoạn hiện tại, làm artifact bắt buộc, thu bằng chứng, tự trả lời Gate rồi mới chuyển tiếp.

Mục tiêu và nguyên tắc sử dụng

Mục tiêu không phải là học nhiều framework nhất. Mục tiêu là hình thành khả năng hiểu hệ thống từ nghiệp vụ đến production, phát hiện failure mode trước khi sự cố xảy ra, thiết kế thay đổi có test/metric/rollback và review dựa trên bằng chứng.

Senior không phải người “biết hết”. Senior biết điều gì quan trọng, điều gì chưa biết, rủi ro nằm ở đâu và cần bằng chứng nào trước khi quyết định.

Đích đến

- Viết được invariant kiểm chứng được.
- Thiết kế đúng trust boundary và authorization.
- Hiểu transaction, retry, duplicate, ordering.
- Định lượng capacity, latency và reliability.
- Deploy, quan sát, rollback và restore.
- Review code/design chưa quen theo risk.

Phân bổ học tập

- **60%** xây dựng, debug, test và fault drill.
- **20%** đọc tài liệu cốt lõi.
- **20%** viết lại, review, ADR và postmortem.

Thời lượng gợi ý: 9–15 tháng ở nhịp 8–12 giờ/tuần. Một số giai đoạn có thể song song; Gate quan trọng hơn lịch.

Không dùng thời gian làm điều kiện qua giai đoạn

Xem hết video, đọc hết sách hoặc viết nhiều code không chứng minh mastery. Chỉ artifact, test, measurement, failure drill và khả năng giải thích/review mới được tính là bằng chứng.

Hệ đo mastery và cổng qua giai đoạn

Thang M0–M4

Mức	Năng lực	Bằng chứng tối thiểu
M0	Chưa biết hoặc chỉ nhận diện thuật ngữ.	Chưa thể giải thích đúng bản chất.
M1	Định nghĩa được “nó là gì”.	Trả lời lý thuyết, chưa áp dụng được.
M2	Áp dụng được khi có hướng dẫn.	Hoàn thành ví dụ/bài tập có sẵn.
M3	Tự áp dụng, test và debug.	Có implementation, test, metric hoặc thí nghiệm tái lập.
M4	Thiết kế, review, dạy lại và phân tích trade-off.	Phát hiện lỗi trong thiết kế khác và bảo vệ quyết định bằng evidence.

Công thức scorecard

Thành phần	Cách tính	Trọng số
Knowledge — K	Tổng mức M đạt được / mức M4 tối đa.	25%
Practice — P	Artifact bắt buộc được hoàn thành và review.	35%
Verification — V	Test, scenario, load/fault/restore drill đạt yêu cầu.	25%
Explanation & Review — R	Gate, design defense và review exercise.	15%

Stage Score = 0,25K + 0,35P + 0,25V + 0,15R

Hard gate — điểm cao không được bù cho lỗi hồng critical

Chỉ được qua giai đoạn khi: 100% mục Critical đạt ít nhất M3; ít nhất 80% toàn bộ mục đạt M3; artifact bắt buộc hoàn thành; Gate đạt từ 80%; không câu Critical nào dưới 2/3; không còn P0/P1 chưa giải thích hoặc chưa có kế hoạch kiểm soát.

Chấm câu hỏi Gate

Điểm	Tiêu chí
0	Không trả lời được hoặc sai bản chất.
1	Nêu được lý thuyết nhưng không áp dụng được vào hệ thống.
2	Áp dụng đúng vào tình huống cụ thể và nêu được cơ chế.
3	Nêu invariant, assumption, failure mode, trade-off, verification và rollback/blast radius.

Metric có giá trị và metric gây ảo tưởng

Nên theo dõi

- % invariant critical có positive + negative test.
- P0/P1 thoát khỏi review.
- Flaky rate và CI stability.
- MTTD/MTTR trong fault drill.
- Message loss, duplicate, freshness.
- p95/p99 ở target load.
- Restore/rollback thực đo.
- Assumption đã biến thành test/metric.

Không dùng làm mastery

- Số giờ xem video.
- Số khóa học/chứng chỉ.
- Số trang sách.
- Số commit hoặc dòng code.
- Coverage tổng thể không gắn risk.
- Số công nghệ/container/microservice.
- Số bug sửa mà không hiểu root cause.

Bản đồ lộ trình

Giai đoạn 0	Baseline hệ thống, invariant, trust boundary, khóa P0.	1–2 tuần
Giai đoạn 1	Python runtime, network, HTTP/WebSocket và API contract.	4–6 tuần
Giai đoạn 2	Database, transaction, migration, time-series và restore.	5–7 tuần
Giai đoạn 3	Security, identity, object authorization và secrets.	5–7 tuần
Giai đoạn 4	Testing, CI, code quality và review theo severity.	4–6 tuần
Giai đoạn 5	Messaging, realtime, partial failure và distributed systems.	6–8 tuần
Giai đoạn 6	Observability, SRE, performance và capacity.	6–8 tuần
Giai đoạn 7	CI/CD, deployment, supply chain, backup và DR.	5–7 tuần
Giai đoạn 8	Architecture, system design, ownership và mentoring.	8–12 tuần+

Luồng xuyên suốt

Testing, security, documentation và review bắt đầu từ giai đoạn 0. Các giai đoạn riêng chỉ là lúc đào sâu, không phải lúc lần đầu thực hiện.

GIAI ĐOẠN 0

Baseline hệ thống và khóa rủi ro nghiêm trọng

Hiểu actor, trust boundary, data flow và invariant của hệ thống hiện tại trước khi học thêm công nghệ.

1-2 tuần

Đầu ra: system map + risk register

Gate: không còn P0 chưa kiểm soát

Core knowledge checklist

- ☐ **CRITICAL** Phân biệt authentication, authorization và object-level authorization.
- ☐ **CRITICAL** Xác định nguồn identity đáng tin cậy của user/device.
- ☐ **CRITICAL** Trust boundary giữa browser, API, broker, device, MySQL và InfluxDB.
- ☐ **CRITICAL** Viết invariant cụ thể, quan sát và kiểm thử được.
- ☐ Actor, resource và action của từng use case.
- ☐ Data flow của login và telemetry.
- ☐ Secret, credential, PII và public identifier.
- ☐ Severity P0/P1/P2/P3 theo impact.
- ☐ Liveness, readiness và dependency health.
- ☐ Failure mode, blast radius và owner.

Artifact bắt buộc trên hệ thống của bạn

System map

```
Device → MQTT → Backend ingest
          → InfluxDB → Realtime Hub
          → WebSocket → Browser
```

Vẽ thêm data flow login, reset password và cấp quyền thiết bị. Đánh dấu nơi identity được tạo, truyền, kiểm tra và có thể bị giả mạo.

10 invariant tối thiểu

- User chỉ đọc device được cấp quyền.
- Device chỉ ghi telemetry cho chính nó.
- Principal identity thắng payload identity.
- Deactivation vô hiệu quyền trong thời hạn cam kết.
- Restart không tái tạo admin mặc định.
- Production secret nằm ngoài source control.

Authorization matrix mẫu

Actor	Resource	Read	Write	Điều kiện
Anonymous	Mọi resource	Không	Không	—
User	Device được cấp	Có	Không	Active, quyền còn hạn
User	Device người khác	Không	Không	Negative test bắt buộc
Device	Chính nó	Không	Có	Credential + topic ACL hợp lệ
Device	Device khác	Không	Không	Reject identity mismatch
Admin	Admin resources	Có	Có	Session/MFA hợp lệ + audit

Các P0 của repository cần đưa vào risk register

MQTT anonymous; credential admin/device mặc định; password recovery dựa vào CCCD; WebSocket cross-device; telemetry spoofing; secret bị track/build context; HTTPS/WSS chưa mặc định; nguy cơ vòng lặp MQTT PING; migration có nhiều nguồn sự thật.

Metric checklist

- ☐ 100% entry point HTTP, WebSocket và MQTT đã được liệt kê.
- ☐ Mỗi entry point có principal, action, resource và policy.
- ☐ 100% invariant P0 có negative test hoặc reproducible check.
- ☐ 0 production credential mặc định; 0 broker anonymous.
- ☐ 0 secret đã biết trong Git index và Docker build context.
- ☐ Vẽ lại request/message flow end-to-end trong dưới 10 phút.
- ☐ Dependency map có failure impact, detection signal và owner.
- ☐ Risk register có severity, deadline và verification.

Gate questions

- 1 Critical:** Device A gửi payload có `device_id=B`. Server phải lấy identity từ đâu, kiểm tra gì và audit gì?
- 2 Critical:** User đã login nhưng không có quyền device X có được mở WebSocket X không? Policy phải đặt ở đâu?
- 3 Critical:** Vì sao CCCD không thể là yếu tố bí mật để reset password?
- 4** Nếu MQTT broker bị compromise, blast radius đi qua những thành phần nào?

5 InfluxDB down nhưng API process còn sống: `/live` và `/ready` nên phản ánh thế nào?

6 Token xác nhận identity có đồng nghĩa với quyền truy cập mọi resource không?

7 Secret, credential, PII và identifier khác nhau ra sao?

8 Restart backend hiện có những side effect nào?

9 Liệt kê mọi điểm một telemetry message có thể mất, duplicate hoặc bị sửa identity.

10 Ba failure mode có blast radius lớn nhất là gì và signal phát hiện tương ứng?

Tài liệu cốt lõi

MUST OWASP API Security Top 10 — API1, API2, API4, API5

Đọc để map BOLA, broken authentication, resource consumption và function authorization vào entry point thật.

MUST Tài liệu nội bộ repository

ADR, deployment guide và bug log là nguồn để kiểm tra documentation drift so với code/config đang chạy.

MUST C4 Model — System Context và Container

Chỉ dùng hai mức đầu để nhìn actor, container, trust boundary; chưa cần component diagram nếu không giúp ra quyết định.

Python runtime, network và API contract

Hiểu request thực sự chạy thế nào, network thất bại ra sao và thiết kế API có contract ổn định.

4-6 tuần

Đầu ra: request lifecycle + API contract

Gate: idempotency + timeout

Core knowledge checklist

Python và runtime

- ☐ **CRITICAL** Process, thread, coroutine và event loop.
- ☐ **CRITICAL** Blocking I/O và non-blocking I/O.
- ☐ **CRITICAL** `def` và `async def` trong FastAPI.
- ☐ GIL với CPU-bound và I/O-bound workload.
- ☐ DB session/socket/client resource lifecycle.
- ☐ Cancellation, timeout và graceful shutdown.
- ☐ Thread MQTT giao tiếp với asyncio/WebSocket.
- ☐ Khi nào cần worker/process riêng.

Network và API

- ☐ **CRITICAL** DNS → TCP → TLS → HTTP.
- ☐ **CRITICAL** Input validation tại boundary.
- ☐ HTTP method, status code và idempotency.
- ☐ Cookie/header/cache-control/content negotiation.
- ☐ Reverse proxy và trusted proxy headers.
- ☐ CORS không phải authentication.
- ☐ Timeout, deadline, retry, backoff và jitter.
- ☐ WebSocket handshake/ping/pong/reconnect.
- ☐ Error contract và stable error code.
- ☐ Versioning, backward compatibility.
- ☐ Pagination/filter/sort và hard limits.
- ☐ OpenAPI và contract test.

Artifact bắt buộc

- Vẽ request lifecycle: Nginx → ASGI → dependency → SQLAlchemy session → response.
- Vẽ WebSocket lifecycle từ handshake, authentication, revalidation đến disconnect cleanup.
- Đề xuất API namespace `/api/v1` và stable error envelope có `code` + `request_id`.
- Bound mọi list/history endpoint; mô tả khi nào offset hay cursor/time-based pagination.
- Inventory mọi outbound I/O và timeout/deadline tương ứng.
- Xây một create/command endpoint idempotent và test retry sau timeout.
- Đo baseline p50/p95/p99 dưới một workload được mô tả rõ.

```
{
  "error": {
    "code": "DEVICE_ACCESS_DENIED",
    "message": "Không có quyền truy cập thiết bị",
    "request_id": "01J..."
  }
}

total_db_connections =
  replicas × workers_per_replica × pool_size
```

Metric checklist

- ☐ 100% public endpoint có request/response schema.
- ☐ 100% list/history endpoint có hard limit.
- ☐ 100% outbound network/database call có timeout hoặc lý do documented.
- ☐ API contract tests chạy tự động; OpenAPI không có endpoint ngoài ý muốn.
- ☐ 0 raw exception/stack trace trả cho client.
- ☐ 100% domain error quan trọng có stable error code.
- ☐ p50/p95/p99 baseline đi kèm workload, environment và sample size.
- ☐ Connection budget của toàn deployment được tính và so với DB max.
- ☐ Retry test chứng minh không tạo duplicate business effect.

Gate questions

- 1 Critical:** Khi nào dùng `def`, khi nào dùng `async def` trong FastAPI hiện tại?
- 2 Critical:** Một thư viện blocking được gọi trong event loop gây hậu quả gì và đo bằng cách nào?
- 3 Critical:** Client timeout không chứng minh server chưa commit. Thiết kế create request để retry an toàn ra sao?
- 4** `PUT` và `PATCH` khác nhau về contract/idempotency thế nào?
- 5** Vì sao offset pagination có thể chậm hoặc bỏ/nhân bản bản ghi khi dữ liệu thay đổi?
- 6** CORS bảo vệ điều gì và không bảo vệ điều gì?
- 7** Timeout khác deadline thế nào? Timeout của downstream phải quan hệ thế nào với upstream?

- 8 Reverse proxy truyền sai client IP ảnh hưởng rate limiter/audit thế nào?
- 9 WebSocket disconnect phải giải phóng resource và state gì?
- 10 API version mới cần tương thích client cũ theo chiến lược nào?
- 11 Vì sao không trả raw database exception?
- 12 Nếu request bị cancel, transaction và side effect ngoài DB cần xử lý thế nào?

Tài liệu cốt lõi

MUST RFC 9110 — HTTP Semantics

Tập trung §6–10, §9.2.2 idempotency, §13 conditional requests và §15 status codes.

MUST Python — Developing with asyncio

Concurrency & multithreading, running blocking code, debug mode, never-awaited coroutine và exception lifecycle.

MUST FastAPI — Concurrency and async/await

Đối chiếu path operation, dependency và utility function với runtime/thư viện DB thật của project.

Database, transaction và data integrity

Dùng database để bảo vệ invariant; kiểm soát migration, concurrency, time-series và khả năng phục hồi dữ liệu.

5-7 tuần

Đầu ra: versioned migration + restore drill

Gate: concurrency + expand-contract

Core knowledge checklist

Relational database

- ☐ **CRITICAL** PK, FK, unique và check constraint.
- ☐ **CRITICAL** ACID và transaction boundary.
- ☐ **CRITICAL** Isolation level và read phenomena.
- ☐ Lock, deadlock và MVCC.
- ☐ Normalization/denormalization.
- ☐ Composite index và thứ tự cột.
- ☐ Query plan và **EXPLAIN**.
- ☐ N+1 và query budget.
- ☐ Optimistic/pessimistic concurrency.
- ☐ Connection pool sizing.

Migration, time-series và recovery

- ☐ **CRITICAL** Một nguồn migration duy nhất.
- ☐ **CRITICAL** Expand-contract và backward compatibility.
- ☐ Fresh install, upgrade, downgrade/roll-forward test.
- ☐ DDL không chạy trong API startup.
- ☐ Event time, ingest time và clock drift.
- ☐ Retention, downsampling và data lifecycle.
- ☐ Influx tag cardinality và bounded query.
- ☐ Late/out-of-order events.
- ☐ RPO, RTO, PITR và offsite backup.
- ☐ Restore verification ở application level.

Artifact bắt buộc

- Chọn Alembic làm một nguồn schema version; loại runtime DDL khỏi app startup.
- CI test database trống, upgrade từ version trước, app cũ/schema mới trong expand phase và interrupted migration.
- Constraint + concurrent test ngăn duplicate authorization.
- Lưu và giải thích **EXPLAIN** của năm query quan trọng nhất.
- Đo query count của endpoint danh sách user/device và sửa N+1 nếu có.
- Thiết kế retention/downsampling/tag policy cho InfluxDB.
- Restore MySQL/Influx vào môi trường cô lập và chạy verification query/use case.

Metric checklist

- ☐ Một command đưa database trống đến schema hiện tại.
- ☐ Fresh + upgrade migration test chạy trong CI; 0 runtime DDL.
- ☐ 100% invariant dữ liệu critical có constraint hoặc rationale được review.
- ☐ 0 N+1 trên endpoint chính; năm hot query có plan và baseline.
- ☐ Query telemetry luôn có time window + result bound.
- ☐ Cardinality estimate có công thức và growth projection.
- ☐ Restore drill thành công; RPO/RTO thực đo được ghi lại.
- ☐ Migration tương thích với ít nhất phiên bản app trước trong expand phase.

Gate questions

- 1 Critical:** Vì sao "check rồi insert" vẫn tạo duplicate khi concurrency và constraint nào phải bảo vệ?
- 2 Critical:** Transaction boundary của use case cấp quyền thiết bị nằm ở đâu? Side effect MQTT nằm ở đâu?
- 3 Critical:** Thêm cột bắt buộc vào bảng lớn theo expand-contract gồm các bước nào?
- 4** Read committed và repeatable read khác nhau ở read phenomena nào?
- 5** Deadlock có phải luôn là bug? Khi nào retry và retry cần điều kiện gì?
- 6** Index `(user_id, device_id)` có thay thế `(device_id, user_id)` không?
- 7** Vì sao query "using index" vẫn có thể chậm?
- 8** `create_all` khác versioned migration về version, review và rollback ra sao?
- 9** Event time, gateway time và server time phục vụ các câu hỏi khác nhau nào?
- 10** Vì sao location/sensor value có thể là Influx tag nguy hiểm?
- 11** Backup thành công nhưng chưa restore thử có đáng tin không?
- 12** RPO 15 phút và RTO 2 giờ được chứng minh bằng thí nghiệm nào?

Tài liệu cốt lõi

MUST MySQL 8.4 — InnoDB Transaction Model + Optimization

Isolation, consistent read, locking/deadlock; index, EXPLAIN, optimizer và connection optimization.

MUST SQLAlchemy 2.0 — Transactions and Connection Management

Session transaction lifecycle, commit/rollback, savepoint và isolation.

MUST Alembic Tutorial + Autogenerate limitations

Revision/upgrade/downgrade; autogenerate chỉ là bản nháp và migration phải được review/test.

SHOULD Designing Data-Intensive Applications

Ưu tiên Storage & Retrieval, Encoding/Evolution và Transactions; nối theory với failure/concurrency trong repository.

Security, identity và trust boundary

Bảo vệ đúng principal, đúng action, đúng resource trên HTTP, WebSocket và MQTT; quản lý trọn vòng đời credential/session.

5-7 tuần

Đầu ra: shared policy + threat model

Gate: auth matrix negative tests

Core knowledge checklist

- ☐ **CRITICAL** Authentication khác authorization.
- ☐ **CRITICAL** Object-level authorization.
- ☐ **CRITICAL** Bind identity với transport/session.
- ☐ RBAC, ABAC và ownership.
- ☐ Password hashing và credential rotation.
- ☐ Access/refresh token, session và revocation.
- ☐ JWT issuer, audience, expiry, algorithm.
- ☐ One-time password recovery.
MFA cho privileged account.
- ☐ TLS/WSS/MQTTS và certificate lifecycle.
- ☐ Secret manager và key rotation.
- ☐ Least privilege DB/Influx/broker.
- ☐ Input validation + resource exhaustion.
- ☐ Audit log và tamper resistance.
- ☐ MQTT auth + per-topic ACL.
- ☐ CSRF, XSS, CORS và cookie flags.
- ☐ Supply-chain security.
- ☐ Data minimization và PII lifecycle.

Artifact bắt buộc

- Một authorization policy dùng chung cho REST, WebSocket và MQTT.
- Force `device_id` từ authenticated principal/path/topic; reject + audit mismatch.
- Dashboard WebSocket read-only; device uplink write-only.
- Password recovery bằng one-time hashed token có TTL và generic response.
- Session/token bị revoke khi đổi password, deactivate, đổi quyền hoặc credential leak.
- Mosquitto: no anonymous, per-device identity, topic ACL, TLS, rate/payload limit.
- Threat model cho telemetry flow; mỗi threat có mitigation và residual risk.
- Secret/dependency/container scan và runbook rotation.

Metric checklist

- ☐ 100% endpoint/topic/WS route có policy principal × action × resource.
- ☐ 100% ô “Không” trong authorization matrix có negative test.
- ☐ 0 đường spoof `device_id` qua payload.
- ☐ Thời gian vô hiệu quyền sau deactivation được đo và đạt cam kết.

- ☐ 0 static PII làm secret; 0 long-lived query-string token.
- ☐ MQTT authentication + ACL + TLS được test bằng principal khác nhau.
- ☐ Runtime DB/Influx principal không có DDL/admin privilege.
- ☐ 0 known secret trong source, history mới, image và log.
- ☐ 100% admin/security-sensitive action có audit event.
- ☐ 0 P0/P1 từ threat model chưa có mitigation/owner.

Gate questions

- 1 Critical:** JWT hợp lệ nhưng user đọc device người khác là lỗi authentication hay authorization? Vì sao?
- 2 Critical:** Tại sao server không được tin `device_id` trong payload dù payload đã được decode thành công?
- 3 Critical:** Sau đổi password/deactivate, token cũ bị vô hiệu trong thời gian bao lâu và cơ chế gì đảm bảo?
- 4** HS256 và asymmetric signing khác nhau về trust boundary, key distribution và rotation?
- 5** Refresh token rotation phát hiện/ngăn replay thế nào?
- 6** Vì sao query-string token nguy hiểm với logs/history/proxy?
- 7** CORS không ngăn attacker dùng script hoặc server riêng vì sao?
- 8** TLS bảo vệ điều gì; không bảo vệ compromised endpoint/application logic nào?
- 9** Device A được publish/subscribe những topic nào? Viết ACL tối thiểu.
- 10** Password reset response phải generic và token phải one-time vì sao?
- 11** Audit log khác application log về mục đích, dữ liệu và retention?
- 12** Migration principal và runtime DB principal khác quyền thế nào?
- 13** JWT signing key bị lộ: contain, rotate, revoke, notify và verify theo thứ tự nào?
- 14** Rate limit theo IP bị bypass/chặn nhầm ra sao và key nên kết hợp những yếu tố nào?

Tài liệu cốt lõi

MUST OWASP API Security Top 10 2023

Ưu tiên API1, API2, API3, API4, API5, API8 và API9; map mỗi rủi ro vào route/topic thật.

MUST OWASP ASVS 5.0.0

Dùng Level 1 làm baseline kiểm chứng; lọc control liên quan auth/session/authorization/input/crypto/configuration.

MUST RFC 9700 — OAuth 2.0 Security BCP + RFC 8725 — JWT BCP

Token replay/rotation; JWT algorithm, issuer/audience, key confusion và explicit typing.

LOOKUP Mosquitto authentication and access control

Áp dụng trực tiếp cho per-device identity, password file/plugin, topic ACL và TLS.

GIAI ĐOẠN 4

Testing, code quality và review

Biến assumption thành bằng chứng tự động; review correctness, security và reliability thay vì chỉ nhìn style.

4–6 tuần

Đầu ra: test matrix + CI gates

Gate: seeded defect review

Testing là luồng xuyên suốt

Bạn đã phải viết test từ các giai đoạn trước. Giai đoạn này chuẩn hóa chiến lược, tăng độ tin cậy và luyện review—not bắt đầu test từ con số 0.

Core knowledge checklist

- ☐ **CRITICAL** Test behavior/invariant, không khóa implementation detail.
- ☐ **CRITICAL** Unit, integration, contract và E2E.
- ☐ Positive, negative và failure-path test.
- ☐ Deterministic fixture và test isolation.
- ☐ Mock, fake và real dependency.
- ☐ Concurrency và migration test.
- ☐ Property-based/fuzz test.
- ☐ Fault injection.
- ☐ Load, stress, spike và soak.
- ☐ Targeted mutation testing.
- ☐ CI quality gates.
- ☐ Type check, lint, SAST/SCA/secret scan.
- ☐ Review theo risk axes.
- ☐ Severity P0–P3.
- ☐ Small changes và self-review.
- ☐ Flaky test root-cause analysis.

Test suite bắt buộc

Risk	Test tối thiểu	Tầng
Authorization	REST/WS/MQTT cross-device + expired/deactivated principal	Unit + integration
Identity spoofing	Device A payload B phải reject + audit	Integration
Password reset	Expiry, one-time, replay, generic response	Integration
Data integrity	Duplicate authorization concurrency	MySQL thật
Migration	Fresh schema + upgrade từ version trước	MySQL thật
Dependencies	Influx unavailable + MQTT initial/reconnect	Integration/fault
Realtime	Slow WebSocket client + bounded queue	Integration/load
Contract	OpenAPI + error/status compatibility	Contract

```
format → lint → type check → unit  
→ integration → migration/contract  
→ dependency + secret scan → image build
```

Metric checklist

- ☐ 100% critical invariant có positive + negative test.
- ☐ 10 lần CI liên tiếp không flaky; flaky rate mục tiêu dưới 1%.
- ☐ Critical suite đủ nhanh để chạy trên mọi PR; slow/fault suite có lịch rõ.
- ☐ Test thất bại khi cố tình reintroduce bug đã sửa.
- ☐ Migration/concurrency test dùng database thật.
- ☐ Review exercise phát hiện $\geq 80\%$ seeded defects và 100% P0/P1.
- ☐ False-positive rate trong review $\leq 20\%$; comment có evidence + impact + direction.
- ☐ 0 critical escaped defect trong bốn tuần; nếu có phải có regression test + postmortem.
- ☐ Mọi mock có rationale: boundary nào và rủi ro nào vẫn cần integration test.

Gate questions

- 1 Critical:** Unit test pass có chứng minh transaction/locking với MySQL thật không? Cần tăng test nào?
- 2 Critical:** Thiết kế test chứng minh Device A không thể ghi cho B trên cả MQTT và WebSocket.
- 3 Test behavior khác implementation detail ở ví dụ authorization policy thế nào?
- 4 Khi nào mock giúp cô lập, khi nào làm mất failure model quan trọng?
- 5 Vì sao từng test pass riêng nhưng full suite fail? Điều tra theo thứ tự nào?
- 6 Property-based testing phù hợp payload decoder ở invariant nào?
- 7 Retry/idempotency test phải mô phỏng timeout ở trước/sau commit ra sao?
- 8 Coverage cao vẫn bỏ lọt BOLA vì sao?
- 9 Migration phải test fresh, upgrade, mixed-version và interruption thế nào?
- 10 Review comment tốt gồm evidence, invariant, impact, priority và direction ra sao?

- 11 Khi nào finding là P0/P1/P2? Nêu blast radius và urgency.
- 12 Test nào chạy mỗi PR, nightly và pre-release? Dựa vào cost/risk nào?
- 13 Retry flaky test cho đến xanh che giấu vấn đề gì?
- 14 Đối diện diff AI 1.000 dòng: chia nhỏ và review thế nào?

Tài liệu cốt lõi

MUST Google SRE — Testing for Reliability

Types of tests, test-induced failure, load testing, canary và testing in production.

MUST pytest How-to

Fixtures, parametrization, monkeypatch, logging/warnings và failure handling; luôn gắn với invariant.

MUST FastAPI Testing + WebSocket tests

TestClient, dependency override, async tests và WebSocket trong đúng stack hiện tại.

MUST Google Engineering Practices — Code Review

Review design/functionality/complexity/tests/docs; giữ change nhỏ và tránh áp sở thích cá nhân.

Messaging, realtime và distributed systems

Thiết kế telemetry pipeline chịu được duplicate, retry, mất kết nối, out-of-order, client chậm và dependency failure.

6-8 tuần

Đầu ra: event contract + failure drills

Gate: delivery semantics

Core knowledge checklist

- ☐ **CRITICAL** Partial failure.
- ☐ **CRITICAL** At-most-once, at-least-once, "exactly-once".
- ☐ **CRITICAL** Idempotency và deduplication.
- ☐ Ordering boundary và partition key.
- ☐ Retry, exponential backoff, jitter, budget.
- ☐ DLQ và durable spool.
- ☐ MQTT QoS/session/retained/LWT.
- ☐ Acknowledgement timing.
- ☐ Event ID, boot ID, sequence.
- ☐ Event/gateway/server time.
- ☐ Backpressure và load shedding.
- ☐ Slow consumer và head-of-line blocking.
- ☐ Outbox pattern.
- ☐ Shared state và multi-worker/replica.
- ☐ Replication/consistency trade-off.
- ☐ Clock drift và late event.

Event envelope và artifact bắt buộc

```
{
  "schema_version": 1,
  "event_id": "01J...",
  "device_id": "bound-by-server",
  "boot_id": "boot-...",
  "sequence_number": 123,
  "event_time": "2026-07-15T10:00:00Z",
  "gateway_received_at": "...",
  "server_received_at": "...",
  "measurements": {}
}
```

- Central queue và per-client queue đều có bound + policy explicit.
- Send timeout; disconnect slow consumer; metric drop/queue depth.
- Dedup theo event ID hoặc boot ID + sequence.
- Retry có deadline/budget; durable spool/DLQ nếu delivery contract yêu cầu.
- Test broker restart, Influx down, backend restart, duplicate, reorder, client chậm.
- Chạy nhiều process và chứng minh state nào không được chia sẻ.

Capacity profile bắt buộc trước load test

Số device mục tiêu	_____	Message/device/giây	_____
Payload trung bình	_____ bytes	Ingress MB/giây	_____
WebSocket đồng thời	_____	Fan-out/message	_____
p99 freshness mục tiêu	_____	Loss chấp nhận	_____

Metric checklist

- ☐ Queue không vượt bound trong overload test; 0 unbounded collection trên hot path.
- ☐ Duplicate event tạo 0 duplicate business effect.
- ☐ Message loss/duplicate đúng delivery contract đã công bố.
- ☐ p99 telemetry freshness đạt target load.
- ☐ Slow client không kéo latency client bình thường vượt objective.
- ☐ Broker/Influx restart phục hồi theo runbook; recovery time được đo.
- ☐ Reconnect không tạo thundering herd ngoài limit.
- ☐ Soak test 2–4 giờ không có memory growth bất thường.
- ☐ Metrics có drop, duplicate, retry, DLQ, queue depth, reconnect.
- ☐ Identity spoofing vẫn bị chặn ở mọi failure/retry path.

Gate questions

- 1 Critical:** QoS 1 có đảm bảo business effect chỉ xảy ra một lần không?
- 2 Critical:** Tại sao retry phải đi cùng idempotency và retry budget?
- 3 Critical:** Queue không giới hạn phá latency, memory và recovery thế nào?
- 4** Exactly-once là thuộc tính của broker hay toàn workflow?
- 5** `event_id`, `boot_id`, `sequence` giải quyết duplicate/order/reboot ra sao?
- 6** Ack broker trước khi Influx persist có failure mode nào?
- 7** Ack sau persist nhưng ack bị mất thì điều gì xảy ra?

- 8 Khi nào drop oldest/newest, coalesce hoặc reject producer?
- 9 Slow consumer gây head-of-line blocking thế nào và cô lập ra sao?
- 10 Jitter ngăn thundering herd bằng cơ chế gì?
- 11 Late/out-of-order event được lưu, dedup và query thế nào?
- 12 Hai API worker dùng in-memory hub/client ID MQTT cố định gây vấn đề gì?
- 13 Khi nào best-effort đủ, khi nào cần durable queue/spool?
- 14 Kafka có cần cho hệ thống hiện tại không? Nên trigger định lượng.

Tài liệu cốt lõi

MUST Designing Data-Intensive Applications, 2nd edition

Reliability/scalability; replication/partitioning; transactions; partial failure/clocks; stream processing.

MUST MQTT 5.0 OASIS Standard

CONNECT/PUBLISH, session state, QoS flow, retry, flow control và security.

MUST Google SRE — Handling Overload

Capacity, queue, load shedding và tránh cascading failure.

SHOULD MIT 6.5840 Distributed Systems

RPC/concurrency, fault tolerance, Raft, linearizability và distributed transactions; ưu tiên failure-trace/lab.

Observability, SRE và performance

Biết hệ thống đang phục vụ người dùng đến đâu, lỗi ở đâu, còn bao nhiêu capacity và phục hồi thế nào.

6-8 tuần

Đầu ra: SLO + dashboard + game day

Gate: trace + capacity

Core knowledge checklist

- ☐ **CRITICAL** SLI, SLO, SLA và error budget.
- ☐ **CRITICAL** Latency, traffic, errors, saturation.
- ☐ Structured logs và redaction.
- ☐ Request/trace/event correlation.
- ☐ Metrics, logs và traces.
- ☐ RED và USE methods.
- ☐ Histogram, p50/p95/p99.
- ☐ Tail latency.
- ☐ Symptom-based alert.
- ☐ Multi-window burn-rate alert.
- ☐ Runbook và escalation.
- ☐ Incident lifecycle.
- ☐ Blameless postmortem.
- ☐ CPU/memory/I/O/query profiling.
- ☐ Load/stress/spike/soak test.
- ☐ Capacity planning.
- ☐ Graceful degradation/load shedding.
- ☐ Liveness/readiness/startup probe.

Artifact bắt buộc

- Correlation context xuyên MQTT → ingest → InfluxDB → WebSocket.
- Structured log không chứa token/password/raw PII.
- Dashboard: API RED, DB pool, MQTT reconnect/rate, Influx write, queue/drop, WS clients và telemetry freshness.
- Ít nhất ba user-centric SLO: API availability, persistence success, dashboard freshness.
- Burn-rate alert có owner, severity, threshold và runbook.
- Load profile có target, p95/p99, error/loss, CPU/RAM/disk/queue.
- Fault drills: MySQL slow, Influx down, broker restart, slow client, traffic spike, disk gần đầy.
- Một postmortem có impact, timeline, detection, root cause, contributing factors, actions.

IoT signal set tối thiểu

Flow	Success/Errors	Latency/Freshness	Saturation
HTTP API	status/domain error rate	p50/p95/p99	worker + DB pool
MQTT ingest	decode/reject/write/drop	ingest/write latency	queue + reconnect
InfluxDB	write/query failures	write/query p95/p99	disk/cardinality
WebSocket	connect/disconnect/drop	broadcast/freshness	clients + per-client queue

Metric checklist

- ☐ Một telemetry event được truy vết end-to-end bằng ID.
- ☐ 100% critical flow có success, failure và latency/freshness metric.
- ☐ Dashboard có golden signals + IoT-specific signals.
- ☐ Fault drill bị phát hiện trong MTDD objective.
- ☐ Một người khác xử lý được drill chỉ bằng alert + runbook.
- ☐ MTTR được đo và cải thiện qua ít nhất hai lần drill.
- ☐ Alert tập trung symptom/SLO, không chỉ CPU/RAM.
- ☐ Capacity limit và saturation knee được chứng minh bằng load test.
- ☐ Bottleneck dự đoán được so sánh với bottleneck thật.
- ☐ 0 secret/raw PII trong log sample đã review.
- ☐ 100% postmortem action có owner, deadline, verification.

Gate questions

- 1 Critical:** SLI, SLO, SLA và error budget khác nhau thế nào? Áp dụng cho telemetry freshness.
- 2 Critical:** Vì sao average latency che giấu trải nghiệm xấu? Histogram/percentile cần cấu hình gì?
- 3 Critical:** Alert CPU 80% có thể không actionable; viết alert dựa trên symptom/SLO thay thế.
- 4 Logs, metrics và traces trả lời những loại câu hỏi khác nhau nào?
- 5 Telemetry freshness đo từ timestamp nào khi device clock có thể lệch?
- 6 Error budget ảnh hưởng quyết định release/reliability work thế nào?
- 7 p99 tăng mạnh nhưng p50 ổn định gợi ý điều gì?
- 8 MTDD/MTTR bắt đầu và kết thúc ở mốc nào?
- 9 Health trả 200 nhưng user vẫn không được phục vụ trong tình huống nào?
- 10 High-cardinality label/tag phá metrics/time-series ra sao?
- 11 Load, stress, spike và soak khác nhau về câu hỏi kiểm chứng?

12 Dấu hiệu connection pool là bottleneck và cách phân biệt DB chậm?

13 Runbook tốt cần prerequisites, diagnosis, mitigation, verify và escalation nào?

14 Blameless có đồng nghĩa không có accountability không?

Tài liệu cốt lõi

MUST Google SRE core: SLO, Golden Signals, Alerting on SLOs

User-centric SLI, error budget, latency/traffic/errors/saturation và actionable burn-rate alerts.

MUST OpenTelemetry Observability Primer

Metrics/logs/traces, span/context propagation và correlation; áp vào một telemetry event thật.

MUST Google SRE — Cascading Failures

Load shedding, deadline, retry, resource exhaustion và thiết kế khi dependency chậm.

CI/CD, deployment, backup và disaster recovery

Deploy thay đổi nhỏ bằng artifact bất biến, quan sát được, có đường lui và phục hồi dữ liệu theo RPO/RTO.

5-7 tuần

Đầu ra: delivery pipeline + restore drill

Gate: rollback/roll-forward

Core knowledge checklist

- ☐ **CRITICAL** Immutable/reproducible artifact.
- ☐ **CRITICAL** Rollback và roll-forward.
- ☐ **CRITICAL** Backup khác replication.
- ☐ Multi-stage image/build context.
- ☐ Build/runtime secret handling.
- ☐ Digest, lockfile và provenance.
- ☐ SBOM, SCA, image scanning.
- ☐ Config/environment separation.
- ☐ Rolling/blue-green/canary.
- ☐ Feature flag.
- ☐ Migration sequencing.
- ☐ Graceful shutdown/readiness.
- ☐ Resource/log limits.
- ☐ Infrastructure as Code.
- ☐ Backup/PITR/restore.
- ☐ Deployment audit/change failure rate.

Artifact bắt buộc

PR checks → immutable signed image → staging
→ smoke/integration tests → canary
→ SLO observation → promote hoặc rollback

- Thu hẹp Docker build context; image không chứa `.env`, `.git`, private key, passwd, `.venv`.
- Tag image bằng commit + deploy theo digest; tạo SBOM và scan.
- Secret được inject lúc runtime từ deployment platform/secret manager.
- Staging đủ giống production cho migration/integration/fault smoke test.
- Canary release có success criteria từ SLO.
- Rollback drill và mixed-version migration test.
- Automated offsite backup + alert backup age/failure.
- MySQL/Influx restore drill có application verification.

Metric checklist

- ☐ Môi trường sạch deploy tự động từ một source revision.

- ☐ Artifact truy vết được đến commit/build/provenance và không chứa secret.
- ☐ Rollback đạt time objective; roll-forward path documented.
- ☐ Restore đạt RPO/RTO và pass business verification.
- ☐ 100% backup job có monitoring; backup age trong objective.
- ☐ Critical vulnerability có owner/SLA và evidence đóng.
- ☐ Migration không vượt downtime objective và tương thích mixed version.
- ☐ Readiness chặn traffic vào instance chưa sẵn sàng.
- ☐ Graceful shutdown không mất message ngoài delivery contract.
- ☐ 100% deploy có actor, artifact, time, result và rollback record.

Gate questions

- 1 Critical:** Vì sao xóa secret trong commit mới không đủ nếu từng push? Rotate và purge khác nhau thế nào?
- 2 Critical:** Replication không thay backup trong delete/corruption/ransomware ra sao?
- 3 Critical:** Migration không tương thích làm rollback app thất bại thế nào? Thiết kế expand-contract.
- 4 Tag **latest** làm mất tính truy vết/reproducibility thế nào?
- 5 Secret trong build argument/layer/cache bị trích xuất ra sao?
- 6 Readiness khác liveness khi rolling deploy và dependency outage?
- 7 Canary được quyết định promote/rollback bằng SLI nào?
- 8 Feature flag giảm blast radius nhưng tạo debt/risk gì?
- 9 Khi nào rollback nguy hiểm hơn roll-forward?
- 10 SBOM/provenance/signature giải quyết những câu hỏi khác nhau nào?
- 11 RPO/RTO phải được chứng minh bằng restore drill thế nào?
- 12 Restore kỹ thuật thành công nhưng dữ liệu nghiệp vụ sai: verification gì cần chạy?
- 13 Graceful shutdown của HTTP/MQTT/WS/queue phải phối hợp thế nào?
- 14 Kubernetes có phải điều kiện để có CI/CD tốt không? Khi nào mới đáng dùng?

Tài liệu cốt lõi

MUST Docker Build Best Practices + Build secrets

Build context, layer/cache, multi-stage, digest, secret mount và image hygiene.

MUST NIST SSDF SP 800-218

Prepare, Protect, Produce Well-Secured Software và Respond to Vulnerabilities; dùng làm delivery/supply-chain baseline.

MUST Google SRE — Production Services Best Practices

Ownership, monitoring, capacity, change management, emergency response và production readiness.

LOOKUP InfluxDB Backup and Restore

Chuyển backup từ ghi chú thủ công thành job, retention, monitoring và restore verification.

Architecture, system design và năng lực senior

Đưa ra quyết định có căn cứ, review được code/design chưa quen và chịu trách nhiệm cho một service từ ý tưởng đến incident.

8–12 tuần và liên tục

Đầu ra: 3 architecture options

Gate: design defense + game day

Core knowledge checklist

- ☐ **CRITICAL** Functional/non-functional requirements.
- ☐ **CRITICAL** Invariant, assumption và trade-off.
- ☐ **CRITICAL** Failure mode, blast radius, rollback.
- ☐ Modularity, cohesion và coupling.
- ☐ Dependency direction.
- ☐ Modular monolith vs microservices.
- ☐ Bounded context/context map.
- ☐ Sync vs async communication.
- ☐ Cache consistency/invalidation.
- ☐ Capacity/cost model.
- ☐ Failure domains/SPOF.
- ☐ Replication/partition/consistency.
- ☐ Operational complexity.
- ☐ ADR/design doc/risk register.
- ☐ Service ownership/on-call.
- ☐ Code/design review.
- ☐ Incident leadership.
- ☐ Mentoring và technical communication.
- ☐ Biết nói “chưa cần”.

Capstone thiết kế ba phương án

Phương án	Mục đích học	Trigger phù hợp
1. Production-ready single-node modular monolith	Làm chắc security, testing, observability, backup và delivery.	Quy mô hiện tại; một team; load còn trong headroom.
2. Nhiều API replica + ingest worker riêng	Tách lifecycle/scaling/failure domain của HTTP và telemetry.	API/ingest load khác nhau; cần scale API; in-memory state là bottleneck.
3. Event-driven + durable broker + consumers	Replay, nhiều consumer, durable ingestion và independent scaling.	Delivery requirement, throughput, replay/consumer count đã được đo.

Mỗi phương án phải có: requirements/assumptions, actor/invariant, context/container/data-flow, capacity, data ownership, consistency, failure/security boundary, SLO, deployment/migration, backup/restore, cost/operational burden, trigger chuyển cấp và fallback.

Metric checklist

- ☐ 100% quyết định lớn có assumption, alternatives, trade-off và consequence.
- ☐ Capacity prediction được so với load test; sai số được giải thích và model cập nhật.
- ☐ 100% seeded P0/P1 được phát hiện trong design/code review.
- ☐ Review comment tập trung risk/correctness hơn style.
- ☐ 100% production change có verification + rollback/roll-forward.
- ☐ Nêu được lý do chưa dùng microservices/Kafka/Kubernetes bằng evidence.
- ☐ Có trigger định lượng để tách ingest worker hoặc thêm durable broker.
- ☐ Người khác triển khai được thiết kế từ spec/ADR mà không cần tác giả “dịch miệng”.
- ☐ 100% incident action có owner/deadline/verification; repeat incident giảm.
- ☐ Mentor một developer hoàn thành module mà không viết thay.
- ☐ Escaped defects giảm qua ít nhất ba chu kỳ review.
- ☐ Danh sách unknown/assumption chưa chứng minh luôn được duy trì.

Gate questions

- 1 Critical:** Khi nào modular monolith tốt hơn microservices cho chính hệ thống này?
- 2 Critical:** Trigger nào chứng minh ingest phải tách API? Metric và failure boundary nào?
- 3 Critical:** Một system design phải mô tả failure mode, detection, mitigation và recovery nào?
- 4 Cache làm correctness/invalidation/staleness khó hơn thế nào? Khi nào chưa nên cache?
- 5 Redis, RabbitMQ/NATS và Kafka giải quyết các nhu cầu khác nhau nào?
- 6 Service boundary nên dựa trên domain, ownership, scaling và failure ra sao?
- 7 Shared database giữa microservices tạo coupling gì?
- 8 Scale process hiện tại làm MQTT subscriber/in-memory hub/rate limiter sai thế nào?
- 9 Khi nào strong consistency quan trọng hơn availability?
- 10 Tổng DB connection budget thay đổi ra sao khi tăng replica/worker?

11 ADR tốt ghi context, decision, alternatives, consequences và revisit trigger thế nào?

12 Khi nào optimization là premature; bằng chứng nào cho phép tối ưu?

13 Senior review khác tìm syntax/style ra sao?

14 Không thể rollback database: roll-forward plan và emergency mitigation cần gì?

15 Error budget cân bằng reliability với feature velocity thế nào?

16 Evidence nào chứng minh kiến trúc đơn giản không còn đủ?

Tài liệu cốt lõi

MUST Eric Evans — DDD Reference

Ubiquitous Language, Bounded Context, Context Map, Aggregate, Repository và Domain Service; tránh biến CRUD thành ceremony.

MUST C4 Model

Context + Container là mặc định; Component/Dynamic/Deployment chỉ khi giúp ra quyết định hoặc vận hành.

MUST Google Engineering Practices — Code Review

Code health, functionality, complexity, tests, docs, naming và small CLs; review bằng evidence, không bằng sở thích.

SHOULD Fundamentals of Software Architecture + Software Architecture: The Hard Parts

Dùng để luyện trade-off analysis, architecture characteristics và distributed data/operational decisions.

Hệ thống theo dõi tiến độ

Dashboard qua giai đoạn

Giai đoạn	K	P	V	R	Score	Hard gate	Evidence link
0 — Baseline						☐ Pass	
1 — Runtime/API						☐ Pass	
2 — Data						☐ Pass	
3 — Security						☐ Pass	
4 — Testing						☐ Pass	
5 — Distributed						☐ Pass	
6 — SRE						☐ Pass	
7 — Delivery/DR						☐ Pass	
8 — Architecture						☐ Pass	

K = Knowledge, P = Practice artifact, V = Verification, R = Explanation/Review. Score ≥ 80 không thay thế hard gate.

Weekly learning review

Tuần	Phase / mục tiêu	M trước → sau	Artifact / evidence	Failure đã thử	Unknown / next action
—					
—					
—					
—					
—					
—					

Câu hỏi tự review mỗi tuần

- Tôi đã biến assumption nào thành test?
- Tôi thử failure path nào?
- Điều gì khác dự đoán ban đầu?
- Bug nào tôi sửa nhưng chưa hiểu đủ?
- Metric nào chứng minh tốt hơn?
- Nếu deploy sai, rollback thế nào?
- Tôi giải thích được mà không mở code không?
- Tôi review code/design nào?
- Tôi bỏ sót gì trong review đầu?
- Quy tắc tư duy mới rút ra?

Monthly outcome dashboard

Metric	Baseline	Mục tiêu	Kết quả	Bằng chứng / nhận xét
Core concepts đạt M3/M4				
Critical invariants có +/- tests		100%		
CI stable / flaky rate		≥99% / <1%		
Critical escaped defects		0		
Seeded P0/P1 review recall		100%		
p95 API @ target load				
p99 telemetry freshness				
Message loss / duplicate effect		Contract / 0		
Queue vượt bound		0		
MTTD / MTTR fault drill				
Restore success / RPO / RTO		100% drill		
Change có rollback plan		100%		
Assumption chưa kiểm chứng		Giảm dần		

Kế hoạch khởi động 30 ngày

Tuần	Trọng tâm	Đầu ra không thương lượng
1	System map + P0 containment	Entry-point inventory, authorization matrix, 10 invariants, risk register; đóng MQTT anonymous và rotate credential/secret.
2	Security regression safety net	REST/WS/MQTT negative tests cho cross-device và spoofing; disable default seed/reset flow không an toàn.
3	Runtime/API foundations	Request lifecycle, outbound timeout inventory, hard limits, stable error model, OpenAPI inventory.
4	Migration + CI baseline	Migration strategy một nguồn, fresh/upgrade test plan, CI format/lint/type/unit/integration skeleton.

Evidence pack và checklist review

Evidence pack để qua mỗi giai đoạn

1. One-page summary

Học gì, invariant chính, failure mode mới thấy, trade-off lớn nhất.

2. Implementation

PR/diff nhỏ, focused và không chứa thay đổi ngoài scope.

3. Test evidence

Happy, negative, failure, concurrency/load khi liên quan.

4. Operational

Log/metric/trace, dashboard query, runbook hoặc drill result.

5. Decision

Assumption, options, rationale, consequences, rollback.

6. Self-review

Điều gì có thể sai, phần AI tạo, bằng chứng, giải pháp đơn giản hơn.

Checklist review mọi backend change

Context & correctness

- ☐ Problem/acceptance criteria rõ.
- ☐ Actor, action, resource rõ.
- ☐ Invariant và assumption được ghi.
- ☐ Happy/boundary/empty/null/timezone.
- ☐ Concurrency/retry/duplicate được xét.
- ☐ Error không bị nuốt.

Security

- ☐ Authentication + object authorization.
- ☐ Identity không lấy từ input không tin cậy.
- ☐ Input/resource có limit.
- ☐ Không lộ PII/secret.
- ☐ Least privilege.
- ☐ Audit event khi cần.

Data & reliability

- ☐ Transaction boundary/constraint đúng.
- ☐ Index/query plan được đánh giá.
- ☐ Migration compatible + fresh/upgrade test.
- ☐ Timeout + bounded retry/idempotency.
- ☐ Queue/buffer có bound.
- ☐ Slow consumer/restart/shutdown rõ.

Performance & operability

- ☐ Result bound; không N+1.
- ☐ Connection/resource budget.
- ☐ Tối ưu dựa trên measurement.
- ☐ Structured log + correlation.
- ☐ Success/failure/latency metrics.
- ☐ Alert/runbook cập nhật nếu cần.

Delivery

- ☐ CI xanh và artifact reproducible.
- ☐ Backward compatibility.
- ☐ Rollout/rollback/roll-forward.
- ☐ Không secret trong source/image/log.
- ☐ Docs/ADR/runbook cập nhật.

Review comment chuẩn

[P1] Tiêu đề có impact

Invariant → evidence/location → failure/blast
radius → direction sửa → test cần chứng minh.

Protocol làm việc với AI / vibe coding

AI có thể tạo code. Kỹ sư vẫn phải sở hữu invariant, quyết định, bằng chứng và hậu quả production.

Yêu cầu → invariant + acceptance criteria
→ threat/failure analysis → diff nhỏ
→ tests → static/security scans
→ human/adversarial review
→ staging/canary → quan sát SLO
→ promote hoặc rollback

- Hiểu từng phần code trước merge.
- AI liệt kê assumption/failure mode.
- Auth/transaction/migration/concurrency có test.
- Không gửi production secret cho AI.
- AI không tự quyết destructive migration.
- Diff đủ nhỏ để review.
- Có adversarial review độc lập.
- Giải thích AI được so với code thật.
- Dependency/version kiểm bằng official docs.
- Sau deploy có metric xác nhận behavior.
- Generated migration được review thủ công.
- AI output được coi là untrusted PR.

Dấu hiệu phải dừng merge

Không thể giải thích code; diff quá lớn; auth/transaction side effect không có test; dependency không rõ nguồn; destructive migration; secret xuất hiện; retry không có idempotency; queue/resource không có bound; không có rollback.

Capstone và mốc tốt nghiệp

Production Readiness Release v1

Capstone không phải “thêm feature”. Đó là một release chứng minh hệ thống hiện tại đã chuyển từ MVP sang production-ready modular monolith theo yêu cầu thực tế.

- ❑ Authorization thống nhất REST/WS/MQTT; auth matrix negative tests đầy đủ.
- ❑ 0 default credential, anonymous broker, tracked secret và insecure recovery.
- ❑ Versioned migration; fresh + upgrade + mixed-version tests.
- ❑ Critical test suite + CI gates ổn định.
- ❑ Telemetry contract, identity binding, bounded queue, dedup/delivery contract.
- ❑ SLO, dashboard, alerts, correlation và runbooks.
- ❑ Load/soak/fault tests có target và evidence.
- ❑ Offsite backup + restore drill đạt RPO/RTO.
- ❑ Immutable deployment + canary + rollback/roll-forward drill.
- ❑ Architecture/design defense được reviewer độc lập chấp nhận; 0 seeded P0/P1 bị bỏ sót.

Dấu hiệu đã có nền tảng tư duy senior

- Mô tả data flow và trust boundary.
- Review code lạ và phát hiện P0/P1.
- Thiết kế authorization đa transport.
- Giải thích transaction/retry/idempotency.
- Dự đoán bottleneck rồi load-test.
- Trace một telemetry event end-to-end.
- Xử lý dependency failure bằng runbook.
- Restore trong RPO/RTO.
- Deploy migration compatible.
- Quyết định monolith/tách worker bằng metric.
- Viết ADR/SLO/postmortem tốt.
- Mentor mà không làm thay.

Không có “điểm kết thúc” tuyệt đối

Dấu hiệu senior là biết nói rõ điều mình chưa biết, thiết kế thí nghiệm để tìm câu trả lời, giảm rủi ro cho người dùng/đội ngũ và làm hệ thống an toàn hơn sau mỗi thay đổi.

Danh mục tài liệu cốt lõi — thứ tự ưu tiên

Danh sách này cố ý ngắn. Hoàn thành đầu ra thực hành của từng tài liệu quan trọng hơn việc đọc nhiều nguồn.

Giai đoạn	Must learn	Đầu ra kiểm chứng
Runtime/API	RFC 9110 ; Python asyncio-dev ; FastAPI async	Request lifecycle, blocking experiment, idempotent API.
Data	MySQL InnoDB ; SQLAlchemy transaction ; Alembic	Concurrency test, EXPLAIN, fresh/upgrade migration.
Security	OWASP API ; ASVS ; JWT BCP	Auth matrix, threat model, token lifecycle tests.
Testing	pytest ; FastAPI testing ; SRE reliability testing	Risk-based suite, failure tests, seeded defect review.
Distributed	DDIA ; MQTT 5 ; Handling Overload	Delivery contract, dedup, bounded queue, fault drills.
SRE	Google SRE SLO ; Golden Signals ; OpenTelemetry	SLO, dashboard, alerts, trace và game day.
Delivery	Docker Build ; NIST SSDF ; Production Best Practices	Immutable pipeline, SBOM, canary, restore/rollback.
Architecture	DDD Reference ; C4 ; Google Review	Context/container diagram, ADR, design defense.

Ba cuốn sách nên đọc sâu

1. **Designing Data-Intensive Applications** — data, transactions, replication, partial failure và stream processing.
 2. **Release It!** — timeout, stability patterns, overload, circuit breaker, bulkhead và production failure.
 3. **Fundamentals of Software Architecture** — architecture characteristics và trade-off analysis.
- Phiên bản tài liệu: kiểm tra tháng 07/2026. Tài liệu framework nên luôn được đối chiếu với version dependency thực tế của repository.